INT 221 - MVC PROGRAMMING

Unit 6

Laravel's Database Abstraction Layer - Eloquent ORM

Laravel uses an **Object-Relational Mapping (ORM) system called Eloquent**, which provides a high-level, object-oriented way to interact with your database.

What is Eloquent ORM?

It allows you to work with database records as if they were objects in your application code. This means you can perform database operations using PHP classes and methods rather than writing SQL queries.

Benefits of Eloquent ORM:

- **Database Abstraction**: Eloquent abstracts the underlying database system, making it easy to switch between different databases (e.g., MySQL, PostgreSQL, SQLite) without changing your application code.
- **Object-Oriented Approach**: Eloquent models represent database tables as objects, making it intuitive to work with data in your application.
- **Readable and Maintainable Code**: Eloquent simplifies database interactions, resulting in cleaner and more maintainable code.

- **Benefits of Abstraction-> Database Portability:**

  With Eloquent, you define your database schema and queries in a database-agnostic way. This means that you can easily switch between different database systems without rewriting your database code. Laravel will handle the database-specific differences behind the scenes, giving you the flexibility to adapt to changing requirements or server configurations.

Ease of Development:

- Eloquent reduces the complexity of working directly with SQL. You define your database schema using PHP classes, and you query the database using methods on those classes. This approach simplifies the development process, reduces the likelihood of SQL injection vulnerabilities, and makes code more readable and maintainable.

Student CRUD API (MySQL + Eloquent)

PART 1: MODEL (Concept)

A **Model represents a database table**.

Example: Student model → students table

Role of Model:

- Connect to DB
- Perform CRUD
- Handle business logic

STEP 1: Create Laravel Project

composer create-project laravel/laravel student-api
cd student-api
php artisan serve

STEP 2: Configure MySQL Database-> open .env

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=student_db
DB_USERNAME=root
DB_PASSWORD=

- Create a database in MySQL:

```
CREATE DATABASE student_db;
```

STEP 3: Create Model + Migration + Factory -> php artisan make:model Student -mf

This creates:

- Model
- Migration
- Factory

STEP 4: Migration (Create Table)-> database/migrations/...

```
public function up()
{
   Schema::create('students', function (Blueprint $table) {
       $table->id();
       $table->string('name');
       $table->string('email')->unique();
       $table->integer('age');
       $table->timestamps();
   });
}
```

Run Migration -> php artisan migrate ->Table created in MySQL

STEP 5: Model Setup-> app/Models/Student.php

```
<?php
namespace App\Models;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Factories\HasFactory;
class Student extends Model{
   use HasFactory;
   protected $fillable = ['name', 'email', 'age'];
}
```

STEP 6: Factory (Fake Data)->database/factories/StudentFactory.php

```
public function definition(){
   return [
      'name' => fake()->name(),
      'email' => fake()->unique()->safeEmail(),
      'age' => fake()->numberBetween(18, 30),
   ];
}
```

STEP 7: Seeder-> php artisan make:seeder StudentSeeder

database/seeders/StudentSeeder.php

```
use App\Models\Student;
class StudentSeeder extends Seeder{
   public function run(){
      Student::factory()->count(10)->create();
   }
}
```

Run Seeder-> php artisan db:seed --class=StudentSeeder

- 10 fake students inserted

DIFFERENCE: FACTORY vs SEEDER

Feature	Factory	Seeder
Purpose	Generate fake data	Insert data
Usage	Defines structure	Executes insertion
Example	fake()->name()	Student::create()
Role	Data generator	Data loader

Relation:

Factory → creates fake data
Seeder → inserts it into DB

STEP 8: Create API Controller-> php artisan make:controller Api/StudentController

STEP 9: Controller Code ->app/Http/Controllers/Api/StudentController.php

```
<?php
namespace App\Http\Controllers\Api;
use App\Http\Controllers\Controller;
use Illuminate\Http\Request;
use App\Models\Student;
class StudentController extends Controller{
   // GET ALL
   public function index(){
       return Student::all();
   }
```

```
// CREATE
        public function store(Request $request){
        $request->validate([
            'name' => 'required',
            'email' => 'required|email',
            'age' => 'required|numeric'
        ]);
        $student = Student::create($request->all());
        return response()->json([
                    'message' => 'Student created',
                    'data' => $student
        ]);
        }
// GET SINGLE
public function show($id){
        $student = Student::find($id);
        if (!$student) {
                    return response()->json(['message' => 'Not found'], 404);
}
return $student;
        }
```

```
    // UPDATE
    public function update(Request $request, $id){
        $student = Student::find($id);
            if (!$student) {
                    return response()->json(['message' => 'Not found'], 404);
        }
            $student->update($request->all());
                return response()->json([
                'message' => 'Updated',
                'data' => $student
            ]);
    }
    // DELETE
    public function destroy($id){
        $student = Student::find($id);
        if (!$student) {
            return response()->json(['message' => 'Not found'], 404);
        }
        $student->delete();
        return response()->json(['message' => 'Deleted']);
    }
}
```

STEP 10: API Routes-> routes/api.php

```
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\Api\StudentController;

Route::get('/students', [StudentController::class, 'index']);
Route::post('/students', [StudentController::class, 'store']);
Route::get('/students/{id}', [StudentController::class, 'show']);
Route::put('/students/{id}', [StudentController::class, 'update']);
Route::delete('/students/{id}', [StudentController::class, 'destroy']);
```

STEP 11: TEST API (Postman)

1. **POST (Create)**

   POST /api/students

   ```
   {
     "name": "Rabaab",
     "email": "test@gmail.com",
     "age": 22
   }
   ```

2. GET ALL

GET /api/students

3. GET SINGLE

GET /api/students/1

4. PUT (Update)

PUT /api/students/1

```
{
  "name": "Updated",
  "age": 25
}
```

5. DELETE

DELETE /api/students/1

LARAVEL MIGRATION COMMANDS (COMPLETE LIST)

1. Create Migration: php artisan make:migration create_students_table

- Create a new migration file
- Used to define table structure

With table option: php artisan make:migration create_students_table --create=students

Automatically prepares Schema::create()

Modify existing table: php artisan make:migration add_age_to_students_table --table=students

Used to add columns

2. Run Migrations: php artisan migrate

- Executes all pending migrations
- Creates tables in database

3. Rollback Migrations: php artisan migrate:rollback

- Undo last batch of migrations
- **Rollback specific steps:** php artisan migrate:rollback --step=2

4. Refresh Migrations: php artisan migrate:refresh

- Rollback ALL migrations
- Re-run them again
- Useful during development

With seeding: php artisan migrate:refresh --seed

5. Fresh Migration: php artisan migrate:fresh

- Drops ALL tables
- Recreates from scratch
- Faster than refresh

With seeding: php artisan migrate:fresh --seed

6. Reset Migrations: php artisan migrate:reset

- Rollback ALL migrations
- Does NOT re-run them

7. Migration Status: php artisan migrate:status

- Shows which migrations are run or pending

8. Run Specific Migration: php artisan migrate --path=database/migrations/filename.php

- Run a specific migration file

9. Pretend Migration (Dry Run): php artisan migrate --pretend

- Shows SQL queries without executing

10. Seed with Migration: php artisan migrate --seed

- Run migrations + seed data

Query String

A **query string** is data passed in the URL after **"?".**

- **Example:**
  http://127.0.0.1:8000/student?id=1&name=Rabaab

  Here:

  - id=1
  - name=Rabaab

Syntax: ?key=value&key2=value2

Why Use a Query String?

- Pass data via URL
- Filter/search data
- Identify record (like id)

Access Query String in Laravel

In Controller:

$request->query('id');

OR

$_GET['id'];

Access Query String in Laravel

In Controller:

$request->query('id');

OR

$_GET['id'];

Difference between MongoDB and MySQL with laravel

MySQL	MongoDB
Tables	Collections
Rows	Documents
Fixed schema	Flexible schema
SQL queries	JSON-based

Benefits Use MongoDB with Laravel

- Flexible structure
- Faster for large data
- JSON-friendly
- No migrations required

IMPORTANT

- Laravel **does NOT support MongoDB by default**
- We use a package: mongodb/laravel-mongodb

STEP-BY-STEP SETUP

STEP 1: Install Laravel Project: composer create-project laravel/laravel mongo-app

cd mongo-app

STEP 2: Install MongoDB Package: composer require mongodb/laravel-mongodb

STEP 3: Configure Database (.env)

```
DB_CONNECTION=mongodb
DB_HOST=127.0.0.1
DB_PORT=27017
DB_DATABASE=laravel_mongo
DB_USERNAME=
DB_PASSWORD=
```

STEP 4: Configure database.php: config/database.php -> Add:

```
'mongodb' => [
    'driver' => 'mongodb',
    'host' => env('DB_HOST', '127.0.0.1'),
    'port' => env('DB_PORT', 27017),
    'database' => env('DB_DATABASE'),
],
```

STEP 5: Create Model : php artisan make:model Student

app/Models/Student.php

```
<?php
namespace App\Models;
use MongoDB\Laravel\Eloquent\Model;
class Student extends Model{
        protected $connection = 'mongodb';
        protected $collection = 'students';
        protected $fillable = [
                'name',
                'email',
                'age'
        ];
}
```

STEP 6: No Migration Needed

MongoDB automatically creates a collection when data is inserted.

STEP 7: Create Controller: php artisan make:controller StudentController

STEP 8: Controller Example

```
use App\Models\Student;
class StudentController extends Controller{
        public function insert() {
                Student::create([
                    'name' => 'Rabaab',
                    'email' => 'test@gmail.com',
                    'age' => 22
                ]);
         return "Inserted into MongoDB"
    }
    public function getData() {
        return Student::all();
    }
}
```

STEP 9: Routes: routes/web.php

```
use App\Http\Controllers\StudentController;
Route::get('/insert', [StudentController::class, 'insert']);
Route::get('/students', [StudentController::class, 'getData']);
```

STEP 10: Start MongoDB Server: mongod

OUTPUT

Insert: http://127.0.0.1:8000/insert

Output: nserted into MongoDB

Fetch Data: http://127.0.0.1:8000/students

Output:


```
[
 {
   "_id": "abc123",
   "name": "Rabaab",
   "email": "test@gmail.com",
   "age": 22
 }
]
```

HOW IT WORKS-> Laravel Model → MongoDB Collection → Document stored

PROJECT: Student CRUD
(MongoDB + Resource Controller + Query String)

PROJECT IDEA

- Use **MongoDB database**
- Use **Resource Controller**
- Use **Query String (?id=1) instead of route params**
- Perform **CRUD operations**

STEP 1: Create Project: composer create-project laravel/laravel mongo-query-crud

cd mongo-query-crud

STEP 2: Install MongoDB Package: composer require mongodb/laravel-mongodb

STEP 3: Configure .env

```
DB_CONNECTION=mongodb
DB_HOST=127.0.0.1
DB_PORT=27017
DB_DATABASE=student_db
```

STEP 4: Configure database.php

```
'mongodb' => [
   'driver' => 'mongodb',
   'host' => env('DB_HOST'),
   'port' => env('DB_PORT'),
   'database' => env('DB_DATABASE'),
],
```

STEP 5: Create Model: php artisan make:model Student

app/Models/Student.php

```
<?php
namespace App\Models;
use MongoDB\Laravel\Eloquent\Model;
class Student extends Model{
        protected $connection = 'mongodb';
        protected $collection = 'students';
        protected $fillable = ['name', 'email', 'age'];
}
```

STEP 6: Create Resource Controller:

```
php artisan make:controller StudentController --resource
```

STEP 7: Define Resource Route: routes/web.php

```
use App\Http\Controllers\StudentController;
Route::resource('students', StudentController::class);
```

STEP 8: Controller Code:app/Http/Controllers/StudentController.php

```
<?php
namespace App\Http\Controllers;
use Illuminate\Http\Request;
use App\Models\Student;
class StudentController extends Controller {
  // READ ALL
  public function index() {
      $students = Student::all();
      return view('students.index', compact('students'));
  }
```

```
// SHOW FORM
 public function create(){
          return view('students.create');
 }
 // INSERT
 public function store(Request $request){
     Student::create([
         'name' => $request->query('name'),
         'email' => $request->query('email'),
         'age' => $request->query('age')
     ]);
     return redirect()->route('students.index');
 }
 // EDIT FORM (using query string)
 public function edit(Request $request){
     $id = $request->query('id');
     $student = Student::find($id);
     return view('students.edit', compact('student'));
 }
```

```
    // UPDATE
    public function update(Request $request) {
        $id = $request->query('id');
        $student = Student::find($id);
        $student->update([
            'name' => $request->query('name'),
            'email' => $request->query('email'),
            'age' => $request->query('age')
        ]);
        return redirect()->route('students.index');
    }
   // DELETE
    public function destroy(Request $request){
        $id = $request->query('id');
        Student::destroy($id);
        return redirect()->route('students.index');
    }
}
```

STEP 9: Views: php artisan make:view students.index

resources/views/students/index.blade.php

```
<h2>Student List</h2>
<a href="/students/create">Add Student</a>
<table border="1">
    <tr>
        <th>Name</th>
        <th>Email</th>
        <th>Action</th>
    </tr>
@foreach($students as $s)
    <tr>
        <td>{{ $s->name }}</td>
        <td>{{ $s->email }}</td>
        <td>
            <a href="/students/{{ $s->_id }}/edit?id={{ $s->_id }}">Edit</a>
            <a href="/students/{{ $s->_id }}?id={{ $s->_id }}">Delete</a>
        </td>
    </tr>
@endforeach
</table>
```

php artisan make:view students/create

resources/views/students/create.blade.php

```
<h2>Add Student</h2>
    <form action="/students" method="GET">
        Name: <input type="text" name="name"><br>
        Email: <input type="email" name="email"><br>
        Age: <input type="number" name="age"><br>
        <button>Add</button>
    </form>
```

php artisan make:view students/edit

resources/views/students/edit.blade.php

```
<h2>Edit Student</h2>
    <form action="/students/update" method="GET">
        <input type="hidden" name="id" value="{{ $student->_id }}">
        Name: <input type="text" name="name" value="{{ $student->name }}"><br>
        Email: <input type="email" name="email" value="{{ $student->email }}"><br>
        Age: <input type="number" name="age" value="{{ $student->age }}"><br>
        <button>Update</button>
    </form>
```

php artisan serve-> open: http://127.0.0.1:8000

Insert: http://127.0.0.1:8000/students?name=Rabaab&email=test@gmail.com&age=22

Edit: http://127.0.0.1:8000/students/{id}/edit?id=abc123

Update http://127.0.0.1:8000/students/update?id=abc123&name=Updated

Delete http://127.0.0.1:8000/students/{id}?id=abc123

- **Resource Controller normally uses:**

  /students/{id}

  But here we override using:

  ?id=abc123

- **MongoDB ID:**

  _id instead of id

- **Query String Drawback**

  Not secure

  Visible in URL

  Not REST standard